

ORIENTATION

What this is

Two tabs, two jobs

LIVE AUDIT — CHROME

Ticker + nav

Header

Control bar

LIVE AUDIT — WHILE IT RUNS

Status strip

Pipeline (7 stages)

Scrape sources

Console

LIVE AUDIT — THE RESULT

National map

Readout + index

History & IQR charts

Anomalies table

Method drawer

Keyboard shortcuts

ANALYTICS TAB

1 • Time series

2 • Collection status

3 • Fare decomposition

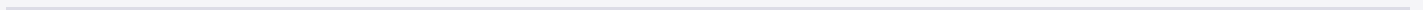
4 • Elasticity curve

5 • Carrier comparison

6 • Sector heatmap

7 • Back-test

8 • API explorer



What every line on the VAYU-SUCHAK dashboard actually means

VAYU-SUCHAK computes a real-time **Airfare Price Index** for India — a Laspeyres index, base 2022 = 100, built from fares scraped live off airline and OTA sites, purified of outliers, checked by two in-house ML models, and rolled up into one number the same way MoSPI's own CPI is built. This page walks every visible element, region by region, top to bottom, and says in plain language what it is, where its number comes from, and — where it matters — whether it's a real computed value or a scripted placeholder.

Two tabs, two jobs

The top-left nav switches between them. Neither depends on the other; you can leave either mid-thought and come back.

Live Audit

default tab

One corridor, one button. Press **Execute Audit** and watch the real pipeline run end to end — scrape, clean, score, index, store — for the route and dates you picked. This is the demo surface: it boots from a calm DORMANT screen into a live instrument panel.

Analytics

second tab

The whole basket at once, from data already sitting in the database — no scrape triggered by opening it. Eight panels: the index over time against the official CPI, collection health, fare composition, price-vs-lead-time curves, carrier comparison, a per-route heatmap, the back-test against MoSPI, and a live API explorer.

Ticker + top nav

very top of the page

Scrolling ticker

thin strip, very top

A marquee of recent per-route sub-index movement (e.g. "DEL–BOM ▲2.1%"). Seeded from the last computed values and refreshed after each audit; on first load with no data yet it's quiet rather than faked.

Live Audit / Analytics

tab switcher

Plain view switch — no data is refetched just by switching back to a tab already loaded once. Keyboard-operable, each button carries `aria-pressed` for the currently active one.

Header

Live Audit tab, top bar

VAYU-SUCHAK

wordmark

The product name. Subtitle underneath: *Real-time Airfare Price Index · MoSPI · SIH26056* — MoSPI is the Ministry of Statistics & Programme Implementation (the CPI's publisher, and this project's real-world validation target); SIH26056 is the Smart India Hackathon problem-statement id this was built against.

f method

header button

Opens the **method drawer** — the actual formulas and live values behind whatever's on screen right now, not marketing copy.

📄 report (PDF) / ⬆ export CSV

header buttons

Appear only after a run completes. Print renders a formatted one-page report of that run's results; export CSV downloads the underlying fare rows the index was built from.

● RUNNING / COMPLETE / ERROR

status dot, right of header

The one true state indicator for the current run — grey/idle before Execute, pulsing while the SSE stream is open, solid green on a clean finish, red if the stream errors or the backend can't be reached.

Control bar

the panel with Execute Audit

The only inputs you set before pressing Execute. Four fields, one button, one mode switch.

Route

origin $\rightleftharpoons$ destination selects

Pick both ends of a corridor from the DGCA-ranked basket (the destination list narrows to that origin's real partners — you can't pick a pair that isn't in the modelled basket). $\rightleftharpoons$ swaps them in place.

Date range

two date fields + presets

The departure window to audit — defaults to today through +29 days. **Diwali $\pm$ 7d / Nov 2026 / Q4 2026** are one-click presets for demoing festival effects. Capped at 100 days; an invalid range (end before start) blocks Execute with an inline reason instead of running anyway.

k_factor

slider, 0.5–5.0

The Tukey-fence multiplier for the outlier filter: `fence = [Q1 - k·IQR, Q3 + k·IQR]`. Lower k is stricter (excludes more fares as anomalies); higher k is permissive. Moving the slider alone changes nothing on screen — it only takes effect on the next Execute, which is deliberate (no silent recompute).

► Execute Audit

the one action button

Opens a live SSE connection to the backend and runs the real 7-stage pipeline (see Pipeline) against your chosen corridor, dates, and k. Disabled and re-labelled "Auditing..." mid-run so it can't be double-fired.

● ● Live scrape /

● ● Cached data

mode toggle

Live scrape (default) hits the real airline/OTA sites fresh, no automatic top-up — if a source is down, it shows as down. **Cached data** reads today's already-collected snapshot with zero network calls to any live source: the one to use for repeat testing or a stage rehearsal so you don't wear out your welcome with the sites you scrape. Sticky across reloads.

† import CSV

next to Execute

Runs a pre-scraped fare extract through the same real pipeline — festival flag, IQR, Laspeyres — via `POST /api/import-csv`, instead of a live scrape. Useful for a corridor or date you can't scrape live right now.

Status strip

six tiles, appears once a run starts

Stages e.g. 4/7	How many of the 7 pipeline stages have completed in this run.
Scrape RTT seconds	Wall-clock time the live-scrape stage actually took — the slowest of the sources that ran, since they fan out concurrently (capped at 40s per source; see Scrape sources).
Throughput rec/s	Fare records ingested per second during acquisition — a real rate computed from records-ingested ÷ ingest time, not a simulated jitter.
Queue	Sources still pending (not yet resolved ok/blocked/degraded) at that instant, read straight off the live per-source SSE stream.
Integrity %	Share of ingested fares that survived both the IQR fence and the ML integrity pass — i.e. made it into the number, unflagged.
Clock	Just the current time — the one tile that isn't about the run.

Pipeline

left column, 7 numbered nodes

Every node here is a real backend stage — the same 7 that run in

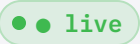
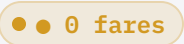
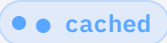
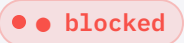
`app/orchestrator.py` and stream one console line each as they finish. A node glows while its stage is in flight, turns solid green with a check when it lands.

#	NODE	WHAT ACTUALLY HAPPENS
1	INGEST	The 3-tier fallback chain runs: live scrape pool → Travelepayouts API top-up if thin → on-disk cache if still thin. Every fare is tagged with which tier produced it.
2	FESTIVAL	Fares within ± 7 days of a gazetted Indian holiday (via the <code>holidays.India()</code> dataset) are flagged, so a later "ex-festival" index can exclude them.
3	IQR	The two-tailed Tukey-fence outlier filter runs at your chosen k . Fares outside $[Q1 - k \cdot IQR, Q3 + k \cdot IQR]$ are excluded and land in the <code>anomalies</code> table.
4	INTEGRITY	An IsolationForest — a second, learned pass on the IQR-clean set — flags fares that are structurally odd (wrong price-per-km, implausible time-to-departure for the price) even though they sat inside the fence.
5	NOWCAST	A gradient-boosted model fills in any route in the basket this run has no clean quote for (never scraped, or every quote for it was sold out) — imputation of a real missing observation, never a forecast of the future.
6	LASPEYRES	The weighted index itself is computed — see the formula under <code>Method drawer</code> .
7	DB	The run is committed — Postgres if reachable, an on-disk SQLite WAL file if not (the readout's DB badge shows which).

real Every node above corresponds to an actual function call in the backend, not a canned animation waiting for a fixed timer — a genuinely slow stage (e.g. a source stuck mid-retry) makes its node visibly stay lit longer, because it is.

Scrape sources below the pipeline

One row per source, live per-source status as each adapter resolves — not just a summary after the whole run finishes.

DOT / TAG	MEANING
 live	Returned real fares this run.
 slow	Returned real fares, but took longer than 6s — noted, not penalised.
 0 fares	Ran, returned nothing usable this run.
 cached	This source wasn't part of the live pool this run (e.g. skipped for speed); shown from its last cached snapshot.
 blocked	robots.txt disallows the fare-search path (IndiGo), an anti-bot wall (MakeMyTrip's Akamai), or a live HTTP block (403/429 that outlasted retrying). Coverage for a blocked airline still comes through Yatra's permitted meta-feed.

Seven targets show by default (Ixigo is deliberately excluded from this panel — robots-disallowed and not part of the demo story). Air India, Akasa and Yatra hit the sites' own JSON APIs directly; Cleartrip and Google Flights are driven by headless Playwright.

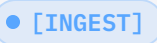
Console right of the pipeline

A real, timestamped log of the run — each line is the literal message the backend sent over SSE for that stage, colour-tagged by which pipeline node it belongs to.

VAYU-SUCHAK KERNEL
boot, before Execute

Five lines (`holidays.India(2026) ... [OK]` , `scraper.pool handshake ... [OK]` , etc.) that always print, always say `[OK]` . This one part of the console is genuinely scripted flavour text, not a real diagnostic — it's cinematic dressing before the real pipeline starts, same idea as a loading screen. Everything from the moment you press Execute onward is real backend output.

Tag colours
[STAGE] prefix on each line

 `[INGEST]`  `[DB]` reference/acquisition stages ·  `[FESTIVAL]`
 `[IQR]`  `[INTEGRITY]` attention/deviation stages ·  `[LASPEYRES]` the
computed result · a row tinted red end-to-end means that line carries a
`[FALLBACK]` notice — a tier below live scrape had to be used.

[+n.nnns]
line prefix

Elapsed time since the run started, accumulating — lets you see exactly how long each stage actually took relative to the others.

National map centre column

City dots
DEL BOM BLR CCU GAU HYD MAA

The seven metros in the modelled basket. Their arcs light up and recolour by that route's own sub-index once a run completes.

Arc colour
green → amber → red

Green = at/near the 2022 base, amber = moderately above it, red = notably inflated relative to base — the same three-tier convention the Analytics heatmap echoes later with an explicit text label alongside the colour.

Selected corridor
legend, thicker glowing arc

The route you actually audited this run, distinguished from the other basket routes shown for context. Hover any arc for that route's sub-index.

Readout right column, top

Big split-flap number

e.g. 118.75

The Airfare Price Index itself — **base 2022 = 100**, so 118.75 reads as "18.75% above the 2022 base fare for this basket." Digits flip mechanically only for the ones that actually changed from the previous value.

ex-festival

secondary readout

The same index recomputed with festival-flagged fares excluded — isolates the underlying trend from seasonal spikes. The chip beside it (▲/▼ + a number) is the festival component: how many index points festivals alone are adding or removing right now.

playwright scrape / cache snapshot

source badge

Which acquisition tier actually produced this run's index — honest about a fallback, not just always saying "live."

database

DB badge

Where the run was actually persisted — `postgres` or `sqlite_fallback` — reflecting the real 3-tier persistence resilience contract, not a fixed label.

History & IQR distribution below the readout

Index History

line chart

The VAYU index (solid) against MoSPI's own published CPI Air Fare (dashed) over past runs/collected days. On the very first observation it says exactly that rather than drawing a misleading single-point line.

IQR Distribution

histogram

The actual fare distribution this run, with the Tukey fence drawn across it — bars past the fence are the ones excluded as anomalies. Dragging `k` on the control bar and re-running visibly moves this fence and re-colours the bars, because it's the same real filter, not an illustration of one.

Anomalies table below the main grid

Only appears when the count is non-zero — at zero it says "no anomalies at k=<value>" instead of showing an empty table. Columns: **Flight**, **Route**, **Fare**, **Reason** (the actual exclusion reason — outside the IQR fence, low or high side, or an ML integrity flag).

note Zero anomalies on a clean day is the outlier test working correctly, not a bug — the fence is meant to catch genuinely odd fares, and most days most fares aren't odd.

Method drawer opened via *f* method

The actual math, not a marketing description of it.

$$\text{Index}_t = [\Sigma(P_t \cdot Q_0) / \Sigma(P_0 \cdot Q_0)] \times 100$$

P_t — median of the fares that passed the IQR fence for this corridor, this run. P_0 — the frozen base-year (2022) median fare. Q_0 — the base-year passenger-volume weight for that route pair, from DGCA traffic data. The basket is fixed at the base period so the index isolates pure price change, the same principle behind any Laspeyres-style CPI.

$$\text{fence} = [Q1 - k \cdot \text{IQR} , Q3 + k \cdot \text{IQR}]$$

Fares outside the fence are excluded before the median is taken; k is the slider value, analyst-set, lower = stricter.

Keyboard shortcuts command bar, bottom edge

 execute  k-factor ± 0.5  cycle corridor  mute  present mode. Enter is suppressed while focus is inside a text field or on a button (so it doesn't fight typing a date or activating a button by its own native Enter/Space handling).

Analytics – the whole basket, from stored data

Eight panels, top to bottom. None of them trigger a live scrape by opening the tab — every number here comes from `fare_quotes`, the database the daily collector and your Execute-Audit runs have already written to.

1 Airfare Price Index – time series

GET `/api/apix + /api/cpi`

The VAYU index (green, solid) plotted against MoSPI's published CPI Air Fare (blue, dashed) on one real date-scaled axis — not two independent charts glued together. CPI publishes at 2012=100; it's rebased here to the **same 2022=100** base VAYU itself freezes P0 at (using the real 12-month 2022 average, ~221.16), so the two lines are honestly comparable in level, not just visually overlaid. CPI is windowed to its most recent 24 of 141 published months so a very recent VAYU point isn't squashed to an invisible sliver against 12 years of history — the chip states the exact truncation. **Daily / Weekly / Monthly** toggles the VAYU series' own granularity.

note CPI genuinely lags real life by months (government publication cadence) — a visible gap between the two lines' right edges is real, not a rendering bug.

2 Collection status

GET `/api/collector/status`

Total quotes / Routes covered / Carriers covered — running totals across all-time collection. **Back-test days** — how many of the 30 calendar days the back-test needs have actually been collected so far (amber until 30/30). **Latest run** — the most recent collector pass, plus how many quotes that day were sold-out (shown but excluded from the index math). Below: real per-source, per-carrier, and per-advance-purchase-window breakdowns for the latest collection day, and a table of the 10 most recent collector runs with their trigger (scheduled/manual), routes, quotes written, and status.

3 Fare decomposition

GET /api/fare-decomposition

What a fare is actually made of, per carrier —  **base fare**  **taxes**  **UDF** (user development fee, an airport levy)  **convenience fee** and other, as a stacked bar sized to real rupee amounts. A route selector narrows it to one corridor. Only Air India, Akasa and Yatra actually populate this breakdown — Cleartrip and Google Flights don't expose it — and the coverage chip says exactly what percentage of priced quotes carry a real breakdown, rather than silently extrapolating for the sources that don't.

4 Lead-time elasticity

GET /api/elasticity/{route}

Average IQR-purified fare at each advance-purchase window — **T+1, T+7, T+15, T+30, T+45** days before departure — for one selected route. This is the "book early, pay less" curve, drawn from real collected quotes at each window, each point labelled with its own sample size (n=). Fewer than 2 collected windows shows an honest "not enough data yet" instead of a broken or empty chart.

5 Carrier comparison

GET /api/apix/latest?scope=carrier:X

Every carrier's own current basket-wide index, sorted highest to lowest, colour-banded green (below 100) / amber (100–150) / red (150+). Each row also states, in plain numbers, how many of the 32 basket routes were actually observed for that carrier versus filled in by the nowcast model — a thin carrier like a regional operator with real data on only 4-5 routes reads very differently from a major carrier observed on 28, and the panel says so rather than presenting every number with equal confidence.

6 Sector-wise heatmap

GET /api/apix?scope=route:X (×32)

All 32 basket routes as a grid, each cell colour-graded by its latest index value and carrying its own day-over-day change (▲/▼). Deliberately labelled **day-over-day**, not the more usual "week-over-week" — with only a couple of collection days banked so far, a real week-over-week number doesn't exist yet, and a route with only one day of data says "no Δ yet" instead of a fabricated 0.00. Each cell also states its price tier in words (below/near/above basket average) — the same colour-plus-label convention the national map legend uses, so the ranking doesn't rely on colour perception alone.

7

Back-test

GET /api/backtest

LIVE — VAYU vs MoSPI CPI Air Fare, month by month, from real collected data once at least one full calendar month exists. Until then it honestly says "0 month(s) of overlapping data" rather than faking a correlation. **HISTORICAL** — a 2022 comparison reconstructed from the Kaggle fare corpus, shown visibly de-emphasised (dimmed, with an explicit "method demonstration only" badge) because that 2022 dataset has no real scrape date, so its booking-window mix is confounded with travel month and its magnitudes aren't trustworthy — it exists to show the validation method works, not as a real result. Stats shown: months compared, month-on-month correlation, and direction agreement (did VAYU and CPI move the same way that month).

8

API explorer

every public GET endpoint

Every endpoint below is real and genuinely callable from this panel — editing a parameter and pressing ► **Run** fires the actual request against this same backend and shows the real HTTP status and JSON response, not a mock. This is also the literal NSO/RBI-consumable API surface the project is meant to expose.

GET /api/apix the index time series — scope, freq, k, limit

GET /api/apix/latest most recent computed point for a scope

GET /api/elasticity/{route} lead-time (APW) curve for one route

GET /api/fare-decomposition base/tax/UDF/convenience split

GET /api/cpi MoSPI CPI air fare + transport series

GET /api/backtest APIx vs CPI Air Fare validation

GET /api/quotes raw fare_quotes rows

GET /api/routes basket + base-year P0/Q0

GET /api/corridors route basket + base period

GET /api/collector/status daily-collector health

GET /api/ml/status integrity + nowcast model eval

vayu-suchak field guide · generated from the live source, corrections welcome